

---

## 100-DAY AI/ML SYSTEM DESIGN CHALLENGE

# Day 03 - AI/ML System Design: Production-Grade AI Customer-Support Assistant

A principal-level design for grounded knowledge retrieval, controlled tool execution, conversation state, uncertainty and escalation, specialized models, production serving, scalability, security, evaluation, and cost.

|     |       |        |        |       |
|-----|-------|--------|--------|-------|
| RAG | TOOLS | MEMORY | SAFETY | SCALE |
|-----|-------|--------|--------|-------|

Standalone technical document for interview revision and public sharing

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1. Executive Summary</b>                                   | <b>5</b>  |
| Core design principles . . . . .                              | 5         |
| System outcome . . . . .                                      | 5         |
| <b>2. Problem Definition and Business Objective</b>           | <b>6</b>  |
| Supported customer journeys . . . . .                         | 6         |
| Functional requirements . . . . .                             | 6         |
| Non-functional requirements . . . . .                         | 6         |
| Scope boundaries . . . . .                                    | 6         |
| <b>3. ML Problem Formulation and Case State</b>               | <b>7</b>  |
| What causes successful resolution? . . . . .                  | 7         |
| Resolution labels . . . . .                                   | 7         |
| Case state machine . . . . .                                  | 7         |
| <b>4. RAG as an Evidence Service</b>                          | <b>8</b>  |
| Why customer support needs retrieval . . . . .                | 8         |
| Four information planes . . . . .                             | 8         |
| Online RAG path . . . . .                                     | 8         |
| RAG does not guarantee groundedness . . . . .                 | 8         |
| RAG, fine-tuning, and long context . . . . .                  | 9         |
| <b>5. Knowledge Ingestion, Freshness, and Chunking</b>        | <b>10</b> |
| Offline knowledge pipeline . . . . .                          | 10        |
| Authority tiers . . . . .                                     | 10        |
| Canonical metadata . . . . .                                  | 10        |
| Structure-preserving parsing . . . . .                        | 10        |
| Document-type-specific chunking . . . . .                     | 10        |
| Parent-child retrieval . . . . .                              | 10        |
| Safe publication . . . . .                                    | 11        |
| <b>6. Hybrid Retrieval, Reranking, and Evidence Assembly</b>  | <b>12</b> |
| Contextual query construction . . . . .                       | 12        |
| Hard filters before similarity . . . . .                      | 12        |
| Lexical plus dense retrieval . . . . .                        | 12        |
| Reranking . . . . .                                           | 12        |
| Evidence assembly . . . . .                                   | 12        |
| Evidence sufficiency outcomes . . . . .                       | 13        |
| Retrieval evaluation . . . . .                                | 13        |
| <b>7. Tool Calling, Authorization, and Controlled Actions</b> | <b>14</b> |
| Separation of responsibilities . . . . .                      | 14        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| Tool registry and gateway . . . . .                             | 14        |
| Risk-tiered execution . . . . .                                 | 14        |
| Safe action flow . . . . .                                      | 14        |
| Idempotency and concurrency . . . . .                           | 14        |
| Unknown outcomes . . . . .                                      | 15        |
| <b>8. Conversation Memory and State</b>                         | <b>16</b> |
| Memory layers . . . . .                                         | 16        |
| Why full-history prompting fails . . . . .                      | 16        |
| State provenance . . . . .                                      | 16        |
| Pending questions and confirmations . . . . .                   | 16        |
| Troubleshooting and multi-issue cases . . . . .                 | 16        |
| Consistency and durability . . . . .                            | 16        |
| Human handoff . . . . .                                         | 17        |
| <b>9. Confidence, Clarification, Abstention, and Escalation</b> | <b>18</b> |
| Value of clarification . . . . .                                | 18        |
| Risk-aware thresholds and calibration . . . . .                 | 18        |
| Escalation triggers . . . . .                                   | 18        |
| <b>10. Training Data and Label Construction</b>                 | <b>19</b> |
| Point-in-time case snapshots . . . . .                          | 19        |
| Data sources . . . . .                                          | 19        |
| Labels by component . . . . .                                   | 19        |
| Hard negatives . . . . .                                        | 19        |
| Bias and feedback loops . . . . .                               | 19        |
| Splitting and leakage control . . . . .                         | 19        |
| <b>11. Models, Architectures, and Loss Functions</b>            | <b>20</b> |
| Model evolution . . . . .                                       | 20        |
| Core losses . . . . .                                           | 20        |
| Representative formulas . . . . .                               | 20        |
| Multi-task option . . . . .                                     | 20        |
| What remains deterministic . . . . .                            | 20        |
| <b>12. Online Serving Architecture and End-to-End Flow</b>      | <b>21</b> |
| Major components . . . . .                                      | 21        |
| End-to-end turn . . . . .                                       | 21        |
| Asynchronous work . . . . .                                     | 22        |
| <b>13. Scalability, Latency, and Cost</b>                       | <b>23</b> |
| Capacity dimensions . . . . .                                   | 23        |
| Independent scaling . . . . .                                   | 23        |
| Illustrative latency budget . . . . .                           | 23        |
| Critical-path optimization . . . . .                            | 23        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| Cost model . . . . .                                          | 24        |
| <b>14. Evaluation, Metrics, and Monitoring</b>                | <b>25</b> |
| Evaluation hierarchy . . . . .                                | 25        |
| Model metrics . . . . .                                       | 25        |
| Business metrics . . . . .                                    | 25        |
| System metrics . . . . .                                      | 25        |
| Safety guardrails . . . . .                                   | 25        |
| Evaluation and rollout . . . . .                              | 25        |
| Problem-specific traces . . . . .                             | 25        |
| <b>15. Security, Reliability, and Failure Handling</b>        | <b>26</b> |
| Prompt injection and data exfiltration . . . . .              | 26        |
| Reliability matrix . . . . .                                  | 26        |
| Operational patterns . . . . .                                | 26        |
| Load and chaos testing . . . . .                              | 26        |
| <b>16. Alternatives, Trade-offs, and Production Evolution</b> | <b>27</b> |
| Major design choices . . . . .                                | 27        |
| Production evolution . . . . .                                | 27        |
| When complexity is justified . . . . .                        | 27        |
| <b>17. Final Architecture and Interview Review Checklist</b>  | <b>28</b> |
| Final data and decision flow . . . . .                        | 28        |
| Principal-level interview checklist . . . . .                 | 28        |

**Design philosophy: begin with the business process and sources of uncertainty. Derive labels, features, models, workflows, and infrastructure from what actually causes a support case to be resolved.**

# 1. Executive Summary

**A production customer-support assistant is not an LLM connected to a vector database. It is a controlled decision system that must understand a request, retrieve approved evidence, access live customer state, perform authorized actions, manage uncertainty, and transfer ownership safely.**

The design separates four responsibilities. RAG answers questions about documented knowledge. Customer APIs return authenticated operational state. Deterministic workflows and tools perform authorized actions. Human support handles judgment, exceptions, risk, and low-confidence cases. The LLM interprets language and synthesizes explanations, but it is never the authority for identity, permission, policy, money movement, or irreversible state transitions.

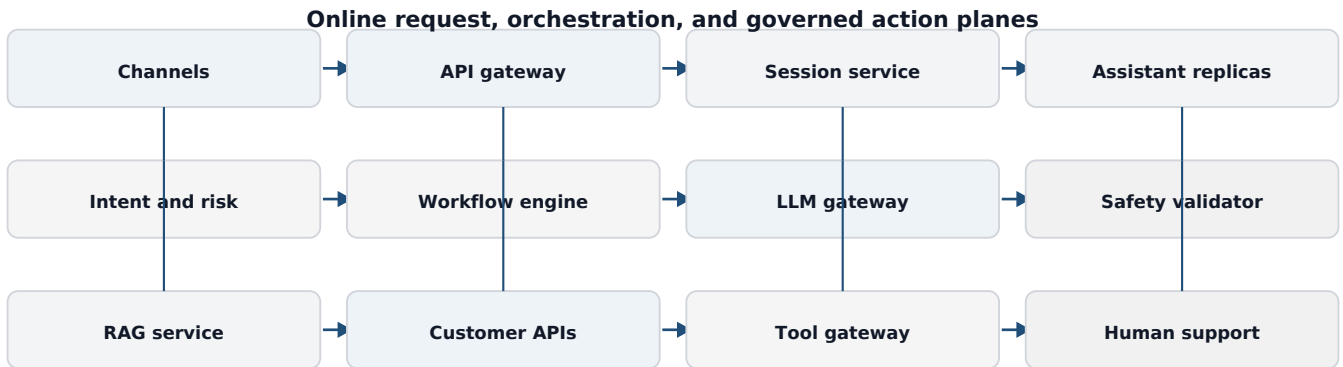


Figure 1. The assistant is an orchestration and decision layer over governed evidence, state, tools, and human support.

## Core design principles

- Optimize safe resolution, not response rate, containment, or conversational fluency in isolation.
- Treat retrieved documents, customer messages, CRM notes, uploads, and tool text as untrusted data.
- Externalize case state so stateless replicas can scale horizontally and any replica can serve any turn.
- Keep structured case state separate from raw conversation history; do not load the full transcript forever.
- Use metadata filters and authority rules before semantic ranking; relevance cannot override authorization or validity.
- Bind confirmations to an exact resource, action, consequence, policy decision, and expiry time.
- Represent ambiguous tool outcomes explicitly as UNKNOWN and reconcile rather than guessing or duplicating an action.
- Evaluate retrieval, reasoning, tools, conversation outcomes, safety, latency, and cost as separate but connected layers.

## System outcome

$$\text{utility} = \text{resolution value} + \text{customer satisfaction} - \text{incorrect-action cost} - \text{customer effort} - \text{human cost}$$

A correct escalation is a successful system decision when automation would be unsafe or unlikely to resolve the case. Conversely, an unhelpful automated answer is not a success merely because the customer did not reach a human.

## 2. Problem Definition and Business Objective

### Supported customer journeys

| Request class   | Example                         | Primary mechanism               | Key risk                       |
|-----------------|---------------------------------|---------------------------------|--------------------------------|
| Knowledge       | What is the return policy?      | RAG over approved policy        | Stale or inapplicable evidence |
| Customer state  | Where is my order?              | Authenticated read-only API     | Cross-customer disclosure      |
| Transaction     | Cancel this order               | Workflow plus authorized tool   | Wrong or duplicate action      |
| Troubleshooting | The red light remains on        | RAG plus stateful procedure     | Repeated or unsafe steps       |
| Mixed           | My order is late; can I cancel? | API plus RAG plus policy engine | Incorrect synthesis            |
| High risk       | Delete my account               | Step-up auth or human review    | Irreversible harm              |

### Functional requirements

- Answer product, policy, troubleshooting, and service questions using current approved evidence.
- Access order, payment, subscription, account, and device state through typed domain APIs.
- Execute approved actions such as cancellation, refund initiation, replacement, or ticket creation.
- Maintain multi-turn and cross-session case continuity without repeating completed work.
- Ask focused clarification, abstain safely, and transfer to the correct human queue with full context.
- Support multiple languages, channels, tenants, products, policy versions, and regional rules.

### Non-functional requirements

| Dimension    | Requirement                                                                | Reason                                                |
|--------------|----------------------------------------------------------------------------|-------------------------------------------------------|
| Correctness  | Evidence-supported claims and verified tool outcomes                       | Fluency cannot compensate for wrong support decisions |
| Safety       | Zero tolerance for unauthorized disclosure and duplicate financial effects | Errors can create direct customer harm                |
| Latency      | Low time to first token and bounded action latency                         | Long waits increase abandonment and retries           |
| Availability | Graceful degradation across model, search, and tool failures               | Support demand often spikes during incidents          |
| Auditability | Reconstruct evidence, policy, model, action, and state transition          | Required for investigation and governance             |
| Scalability  | Independent scaling by RPS, token throughput, search QPS, and tool load    | Different subsystems have different bottlenecks       |
| Privacy      | Data minimization, tenant isolation, retention and deletion controls       | Conversation memory contains sensitive data           |

### Scope boundaries

The assistant may interpret requests, retrieve evidence, summarize, route, and execute allowlisted workflows. It does not directly query operational databases, invent permissions, override policy decisions, access arbitrary network destinations, or claim an action succeeded before a domain service confirms it.

### 3. ML Problem Formulation and Case State

There is no single model target. The system contains related prediction, retrieval, generation, and decision tasks. The architecture should expose each task so that it can be trained, calibrated, evaluated, and rolled back independently.

| Subproblem        | Input X                          | Target y                              | Output use                    |
|-------------------|----------------------------------|---------------------------------------|-------------------------------|
| Intent            | Message plus case context        | One or more intents                   | Workflow and source routing   |
| Entity extraction | Message                          | Spans and types                       | Candidate resource resolution |
| Risk              | Conversation plus verified state | Risk tier                             | Threshold and approval policy |
| Retrieval         | Contextual query                 | Relevant passages                     | Evidence package              |
| Reranking         | Query plus candidates            | Relevance order                       | Top evidence                  |
| Tool selection    | Request plus workflow state      | Tool, no-tool, clarify, escalate      | Action proposal               |
| Grounding         | Claim plus evidence              | Supported, contradicted, insufficient | Validation and abstention     |
| Resolution        | Point-in-time case snapshot      | Probability of automated resolution   | Continue or hand off          |
| Escalation        | Case snapshot plus uncertainty   | Answer, clarify, abstain, escalate    | Decision policy               |

#### What causes successful resolution?

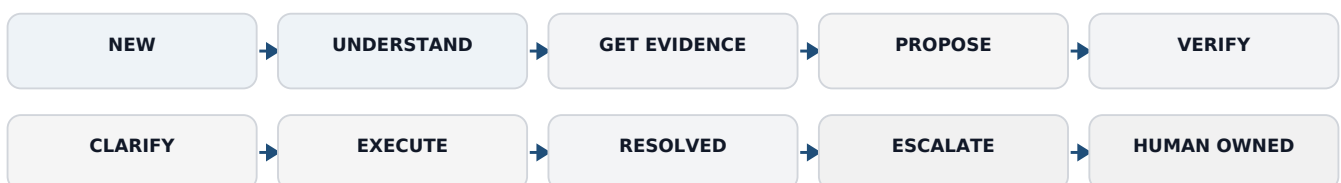
Resolution depends on correctly understanding the goal, having sufficient context, retrieving current evidence, identifying the right resource, selecting the correct workflow, completing backend actions, preserving conversation progress, avoiding repetition, and escalating at the right time. These causes determine which events must be logged and which features are legitimate at prediction time.

#### Resolution labels

- Explicit: the customer confirms the issue is solved.
- Operational: the requested state transition is confirmed by the source-of-truth service.
- Implicit: no same-issue recontact within an issue-specific window.
- Human-confirmed: an agent verifies that automation was correct.
- False resolution: the interaction closes but the customer recontacts for the same issue.

resolved = valid completion signal AND no same-issue recontact within the issue-specific observation window

#### Case state machine



The state machine controls available tools, required information, confirmation, retries, stopping conditions, and training labels. A case can also be WAITING\_FOR\_CUSTOMER, WAITING\_FOR\_EXTERNAL\_SYSTEM, PROVISIONALLY\_RESOLVED, REOPENED, or ABANDONED. Handoff is not equivalent to resolution.

## 4. RAG as an Evidence Service

**RAG is one subsystem. It answers questions about documented knowledge. Live customer state comes from APIs, actions come from tools, and high-risk decisions come from deterministic policy and authorization services.**

### Why customer support needs retrieval

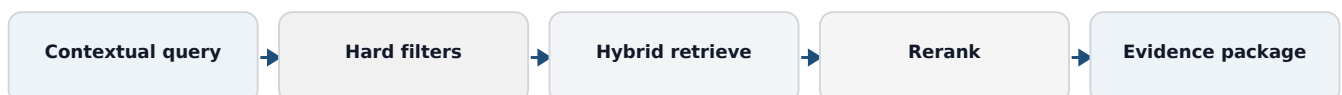
- Support knowledge is organization-specific, versioned, regional, frequently updated, and often permission-sensitive.
- Retrieval allows rapid correction and incremental indexing without retraining a model for every policy change.
- Evidence metadata enables audits: source, version, effective date, audience, access scope, and passage used.
- Long-tail issues may have too few examples for supervised training but can still be answered from documentation.

```
evidence = Retrieve(query, conversation_state, valid_corpus, top_k); answer = LLM(query, state, evidence, policy)
```

### Four information planes

| Plane             | Examples                                  | Access mechanism                 | Freshness                |
|-------------------|-------------------------------------------|----------------------------------|--------------------------|
| Public knowledge  | FAQs, manuals, help articles              | RAG                              | Hours to days            |
| Controlled policy | Refund, warranty, internal playbooks      | Filtered RAG plus policy service | Effective-date sensitive |
| Customer state    | Orders, payments, account, device         | Authenticated domain APIs        | Seconds                  |
| Operational state | Incidents, outages, temporary workarounds | Event-driven index or status API | Near real time           |

### Online RAG path



The evidence service accepts a contextual query, tenant and permission scope, applicability filters, and a token budget. It returns ranked passages, document metadata, coverage, freshness and conflict signals. The orchestrator then decides whether to answer, fetch customer state, invoke a workflow, clarify, abstain, or escalate.

### RAG does not guarantee groundedness

| Failure                | What happens                                 | Control                                    |
|------------------------|----------------------------------------------|--------------------------------------------|
| Retrieval failure      | Correct article is absent                    | Recall tests and hybrid search             |
| Ranking failure        | Correct passage is below irrelevant passages | Cross-encoder reranking                    |
| Context failure        | Exception is truncated or overshadowed       | Structure-aware evidence assembly          |
| Interpretation failure | Model misunderstands a condition             | Claim validation and risk-aware abstention |
| Freshness failure      | Expired policy is selected                   | Hard effective-date filter                 |
| Authorization failure  | Restricted content reaches the prompt        | ACL enforcement before retrieval           |



## **RAG, fine-tuning, and long context**

Fine-tuning is appropriate for behavior such as intent recognition, query rewriting, structured output, tool selection, response style, and escalation. It is a poor primary store for frequently changing facts. Long-context models complement retrieval when the relevant section is large; they do not justify sending the entire corpus on every request.

## 5. Knowledge Ingestion, Freshness, and Chunking

### Offline knowledge pipeline



Source connectors detect new, changed, deleted, activated, expired, or permission-modified content. Preserve an immutable raw copy with source ID, checksum, version, content type, permissions, and ingestion time. This enables reprocessing, audits, and comparison between original and extracted text.

### Authority tiers

| Tier                 | Content                                            | Use                                                           |
|----------------------|----------------------------------------------------|---------------------------------------------------------------|
| 1 - authoritative    | Approved policies, official docs, active incidents | May support final customer claims                             |
| 2 - curated internal | Reviewed playbooks and troubleshooting             | May guide workflow; visibility restricted                     |
| 3 - discovery        | Tickets, forums, draft notes                       | Candidate diagnosis only; cannot independently justify claims |

### Canonical metadata

Every document and chunk should carry document ID, source ID, type, authority tier, owner, version, approval status, effective dates, language, country, product, plan, audience, access scope, parser version, chunking version, embedding version, and checksum. Missing ACL or approval metadata sends content to quarantine; it never defaults to public.

### Structure-preserving parsing

Preserve headings, subsections, procedures, lists, warnings, tables, code blocks, images, and footnotes. A flat text dump can detach a condition from its exception or a table cell from its region header. OCR output and table parsing need explicit quality checks before publication.

### Document-type-specific chunking

| Document        | Chunk unit                       | Must remain together                                  |
|-----------------|----------------------------------|-------------------------------------------------------|
| FAQ             | Question plus complete answer    | Question, answer, applicability                       |
| Policy          | One policy rule                  | Conditions, outcome, exceptions, effective date       |
| Troubleshooting | One coherent procedure           | Symptom, prerequisites, ordered steps, exit condition |
| Manual          | One product feature or operation | Product/version, instructions, warnings               |
| Table           | Row serialized with headers      | Column meaning, region, plan, product                 |
| Incident        | Current incident state           | Service, region, status, workaround, last update      |

### Parent-child retrieval

Index small child chunks for precise matching, then return the corresponding parent section for generation. This separates the best retrieval unit from the best reasoning unit. Structural overlap is preferred to blindly repeating a fixed token percentage. Chunk size is tuned through retrieval recall, exception preservation, answer correctness, context cost, and reranker latency.

## Safe publication

Build a candidate index, validate parser output and ACLs, run a regression query suite, atomically switch the active alias, monitor, and retain the previous version for rollback. Never remove the old active policy until the replacement is confirmed searchable. Incremental and idempotent processing avoids rebuilding unchanged content.

## 6. Hybrid Retrieval, Reranking, and Evidence Assembly

### Contextual query construction

The latest message is often incomplete. Query rewriting uses verified product, version, symptom, locale, current workflow, recent turns, and completed steps to produce a standalone search query. It must preserve facts and avoid inventing entities. Compound questions are decomposed so each sub-question can use the correct source.

### Hard filters before similarity

- Tenant, permission, audience, active status, and effective date are hard filters.
- Product family, language, region, plan, and recency may be hard filters or soft boosts depending on policy.
- Authorization is never a learned ranking preference; an unauthorized passage is ineligible regardless of similarity.

### Lexical plus dense retrieval

Lexical retrieval is strong for error codes, product IDs, versions, and exact phrases. Dense retrieval is strong for paraphrases, vague symptoms, and multilingual semantics. Generate candidate sets independently and fuse ranks.

```
candidates = BM25(query, k1) UNION ANN(embedding(query), k2); RRF(doc) = SUM_r 1 /  
(constant + rank_r(doc))
```

Approximate nearest-neighbor indexes reduce the need to compare against every vector. Exact search is simpler for small corpora; HNSW-style indexes become useful when corpus size or QPS justifies the memory and recall trade-off.

### Reranking

First-stage retrieval optimizes recall and speed. A cross-encoder or learned reranker scores a much smaller candidate set using query-passage interaction plus authority, freshness, applicability, and historical usefulness. An active slightly less similar policy must outrank a highly similar expired one.

```
rank_score = w1*relevance + w2*authority + w3*freshness + w4*applicability +  
w5*resolution_value - w6*conflict
```

### Evidence assembly

- Remove exact and near duplicates; cap redundant chunks from one parent document.
- Preserve headings, table headers, parent context, source version, and passage-level provenance.
- Detect contradictions and apply deterministic precedence: applicability, active version, explicit override, authority.
- Allocate the token budget to sufficient evidence rather than mechanically concatenating top-k results.
- Return coverage, missing facts, conflicts, and sufficiency status alongside passages.

## Evidence sufficiency outcomes

| Status               | Meaning                            | Next action                                     |
|----------------------|------------------------------------|-------------------------------------------------|
| SUFFICIENT           | Every material question is covered | Generate and validate                           |
| PARTIAL              | Some sub-questions lack evidence   | Answer covered parts and clarify or fetch state |
| NEEDS_CUSTOMER_API   | Question concerns live state       | Call authenticated API                          |
| CONFLICTING          | Applicable sources disagree        | Abstain or escalate                             |
| NO_RELIABLE_EVIDENCE | No authoritative support           | Safe abstention                                 |

## Retrieval evaluation

Use Recall@k, MRR, NDCG, context precision, evidence coverage, freshness accuracy, and authorization leakage. The evaluation set must include ambiguous queries, exact identifiers, expired-policy traps, region and version traps, multi-turn ellipsis, conflicting sources, no-answer cases, misspellings, and adversarial content.

## 7. Tool Calling, Authorization, and Controlled Actions

The LLM may propose an action. Deterministic systems authenticate, authorize, validate policy, bind confirmation, execute, reconcile, and audit it.

### Separation of responsibilities

| Control        | Question                                    | Authority                        |
|----------------|---------------------------------------------|----------------------------------|
| Authentication | Who is the user?                            | Identity service                 |
| Authorization  | May this user act on this resource?         | Authorization and domain service |
| Eligibility    | Do current business rules allow the action? | Policy engine                    |
| Confirmation   | Does the user intend this exact action?     | Workflow-bound confirmation      |
| Execution      | Did the state transition occur?             | Source-of-truth domain service   |
| Explanation    | How should the outcome be communicated?     | Grounded LLM generation          |

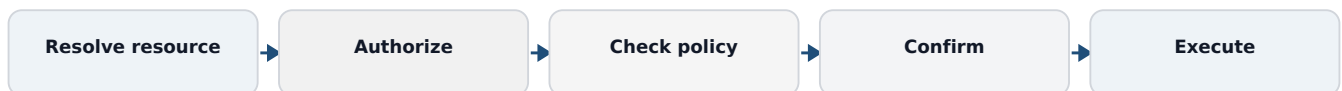
### Tool registry and gateway

Each allowlisted tool has a versioned schema, read/write classification, risk tier, required scope, timeout, retry policy, idempotency requirement, allowed workflow states, confirmation rule, output schema, data sensitivity, and owner. Expose only the small domain-specific tool set relevant to the current workflow to reduce token cost, selection error, and attack surface.

### Risk-tiered execution

| Tier | Example                               | Execution policy                       |
|------|---------------------------------------|----------------------------------------|
| 0    | Search public documentation           | Automatic                              |
| 1    | Read authenticated order status       | Authorization and audit                |
| 2    | Create support ticket                 | Controlled workflow                    |
| 3    | Cancel order or modify subscription   | Eligibility plus explicit confirmation |
| 4    | Refund or security change             | Step-up auth and strict policy         |
| 5    | Large financial or regulated decision | Human approval                         |

### Safe action flow



Entity extraction does not prove resource identity. Resolve references such as 'my last order' through trusted APIs, ask for clarification when multiple candidates remain, and bind the verified internal ID before execution. The confirmation states the exact resource, operation, material consequence, amount if relevant, policy decision, and expiry.

### Idempotency and concurrency

```
idempotency_key = hash(customer_id, case_id, action, resource_id, logical_action_version)
```

A retry with the same key and request returns the original result; the same key with different arguments is rejected. At-least-once request delivery can still provide an effectively-once business outcome through idempotency, unique constraints, valid state transitions, and reconciliation. The domain service atomically

rechecks preconditions during execution, using conditional updates or optimistic concurrency, so a stale eligibility result cannot force an invalid transition.

## Unknown outcomes

A timeout after a refund request does not mean failure. Record UNKNOWN, keep the original operation ID and idempotency key, query the source-of-truth service, and reconcile. Never retry a financial action with a new identity or tell the customer that it failed until the outcome is confirmed. Tool outputs are structured data, not instructions.

## 8. Conversation Memory and State

**Conversation history records what participants said. Structured case state records what is verified, what the workflow permits, what was attempted, and what remains unresolved.**

### Memory layers

| Layer               | Purpose                       | Example                                          |
|---------------------|-------------------------------|--------------------------------------------------|
| Immutable event log | Audit, replay, training       | Message, retrieval, tool, policy, handoff events |
| Case snapshot       | Current verified state        | Intent, resource, workflow, pending action       |
| Recent context      | Local dialogue coherence      | Last relevant turns and pending question         |
| Durable summary     | Compressed older context      | Attempted steps and unresolved issue             |
| Selective history   | Retrieve relevant past events | Previous replacement promise                     |
| Customer profile    | Approved stable preferences   | Language; never inferred permission              |

### Why full-history prompting fails

if each turn adds  $k$  tokens, cumulative conversation processing through turn  $n$  is  $k \cdot n \cdot (n+1) / 2 = O(k \cdot n^2)$

Use a bounded recent window, verified structured state, a faithful summary, and selective retrieval. Evidence and operational state normally deserve token priority over conversational repetition. Summaries never override domain data.

### State provenance

Every material field records value, source, verification state, confidence, timestamp, and supporting event IDs. A customer claim creates an observation; it cannot create an approved policy decision. Resource references move from MENTIONED to CANDIDATE to VERIFIED, and consequential tools require verified entities.

### Pending questions and confirmations

Short replies such as 'yes', 'the first one', or 'I already did' are resolved against explicit pending state. A confirmation object identifies the action, resource, consequences, policy decision, and expiry. When more than one pending question is plausible, ask rather than guess.

### Troubleshooting and multi-issue cases

Store each troubleshooting step, result, source, current branch, and exit condition. Split multiple customer goals into subissues so resolving an order delay does not silently close a duplicate-charge complaint. Maintain channel-independent case state while preserving channel-specific authentication assurance.

### Consistency and durability

- Deduplicate customer messages with client-generated message IDs; separately make tool actions idempotent.
- Use case sequence numbers or optimistic concurrency so two tabs cannot silently overwrite state.
- Append durable events and update the materialized snapshot before reporting a consequential success.
- Represent external actions as PENDING, SUCCEEDED, FAILED, or UNKNOWN and reconcile callbacks.
- Track promises as commitments with owner, due time, SLA, and completion status.



## Human handoff

Transfer the customer goal, verified resources, concise summary, steps and results, evidence versions, tool calls, pending operations, escalation reason, sentiment, commitments, and recommended next action. Once the state becomes HUMAN\_OWNED, the bot does not independently perform consequential actions unless explicitly delegated.

## 9. Confidence, Clarification, Abstention, and Escalation

Confidence is not one model score. The decision combines intent and entity certainty, retrieval coverage, evidence consistency and freshness, grounding, tool selection, policy result, risk, and predicted resolution. A high vector similarity score does not mean an answer is safe.

| Decision          | Use when                                                                 | Desired behavior                              |
|-------------------|--------------------------------------------------------------------------|-----------------------------------------------|
| Answer or execute | Evidence is sufficient, applicable, authorized, and low enough risk      | Respond from evidence or verified tool result |
| Clarify           | One missing detail can make the case solvable                            | Ask the smallest high-information question    |
| Abstain           | Reliable evidence, capability, or permission is absent                   | State the limitation and safe next step       |
| Escalate          | Human judgment, approval, empathy, or privileged investigation is needed | Transfer ownership with full context          |

### Value of clarification

```
value_of_question = expected reduction in decision risk - customer effort of answering
```

Do not ask the customer for information already available through an authorized backend service. Ask a targeted question when intent, resource, version, region, or confirmation is ambiguous. Repeated low-value clarification is a failure mode and should trigger workflow change or escalation.

### Risk-aware thresholds and calibration

```
automate only when P(correct | case_state) >= threshold_for_risk_tier
```

Thresholds rise with the cost of error. Public FAQs may tolerate moderate confidence; account deletion, refunds, or security changes require high confidence plus deterministic controls. Calibrate probabilities using temperature scaling, Platt scaling, or isotonic regression, and measure calibration by intent, language, product, region, and risk tier.

### Escalation triggers

- Explicit request for a human, mandatory policy, safety, legal, fraud, security, or regulatory concern.
- Conflicting evidence, unknown tool outcome, repeated automation failure, or unsupported workflow.
- High distress, repeated misunderstanding, dispute of a prior decision, or low predicted resolution probability.

The resolution model can estimate whether continued automation is likely to succeed, but it must never be used to deny human access. Handoff is recorded separately from resolution and routed by skill, language, product, region, risk, priority, and SLA.

## 10. Training Data and Label Construction

### Point-in-time case snapshots

The core training unit is a support case at a specific decision time. Features contain only information available at that moment. Later human diagnosis, completed refund, or updated policy cannot leak into an earlier prediction.

$X_t$  = information available at decision time  $t$ ;  $y$  = later observed or reviewed outcome

### Data sources

Use conversations, agent dispositions, selected articles, search logs, tool calls, workflow transitions, human edits, resolution codes, satisfaction, and same-issue recontact. Expert-authored synthetic cases improve rare-risk coverage but do not replace real evaluation. Production logs are observations, not automatically correct labels.

### Labels by component

| Component  | Label                                        | Important negatives                                 |
|------------|----------------------------------------------|-----------------------------------------------------|
| Intent     | One or more intents                          | Closely related intent and out-of-scope             |
| Entity     | Spans, types, verified mapping               | Ambiguous and invalid references                    |
| Retriever  | Relevant passage                             | Wrong region/version, expired, incomplete exception |
| Reranker   | Graded relevance or ordering                 | Topically similar but inapplicable                  |
| Tool       | Tool/no-tool/clarify/escalate plus arguments | Unauthorized, ambiguous, missing confirmation       |
| Grounding  | Supported, contradicted, insufficient        | Initiated versus completed outcome                  |
| Resolution | Successful automated resolution              | Recontact, correction, abandonment                  |
| Escalation | Continue, clarify, abstain, escalate         | Unnecessary escalation and unsafe continuation      |

### Hard negatives

Random negatives are too easy. Retrieval training needs same-product wrong-issue passages, same-policy wrong-country versions, expired documents, similar error codes for different devices, and a main rule that omits the relevant exception. Tool data needs prompt injection, nonexistent resources, invalid argument combinations, and unauthorized requests.

### Bias and feedback loops

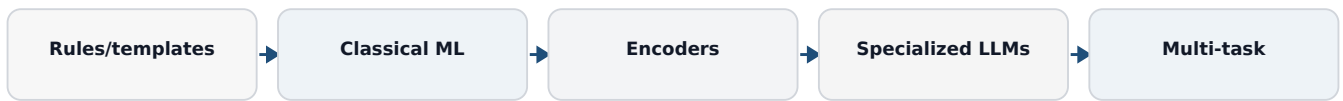
- No repeat contact may mean resolution, abandonment, or channel switching.
- Agents see cases that automation failed, so human-labeled data is selectively sampled.
- Highly ranked documents get more exposure and appear more successful, reinforcing existing rank bias.
- Old conversations encode old policies and workflows; retain policy-version metadata.
- Satisfaction may reflect the underlying business outcome rather than assistant quality.

### Splitting and leakage control

Split by case rather than individual turn, use customer-level separation where appropriate, and prefer temporal test sets. Add product, region, new-intent, and policy-version holdouts to measure generalization. Rare high-risk classes need class weighting, focal loss, oversampling, expert synthesis, and deterministic guardrails.

# 11. Models, Architectures, and Loss Functions

## Model evolution



Begin with measurable baselines. Rules and templates are strong for common deterministic workflows. TF-IDF plus logistic regression provides a transparent intent baseline. Encoder classifiers, dual-encoder retrieval, cross-encoder reranking, and constrained generation are introduced only when they address measured limitations.

## Core losses

| Task                | Model                                   | Loss                                        |
|---------------------|-----------------------------------------|---------------------------------------------|
| Multi-label intent  | Encoder with sigmoid heads              | Binary cross-entropy or focal loss          |
| Entity extraction   | Token or span classifier                | Token/span cross-entropy                    |
| Query rewriting     | Seq2seq or constrained LLM              | Autoregressive token loss                   |
| Dense retrieval     | Dual encoder                            | Contrastive InfoNCE with hard negatives     |
| Reranking           | Cross-encoder or LTR                    | Pairwise logistic or listwise ranking loss  |
| Tool selection      | Classifier or structured generator      | Cross-entropy plus schema constraints       |
| Grounding           | NLI-style classifier                    | Three-class cross-entropy                   |
| Resolution          | Calibrated classifier or survival model | BCE, focal, or censored-time likelihood     |
| Escalation          | Cost-sensitive classifier               | Expected decision cost                      |
| Response generation | Grounded LLM                            | SFT token loss plus preference optimization |

## Representative formulas

$$\text{intent\_BCE} = -\sum_j [y_j \cdot \log(p_j) + (1-y_j) \cdot \log(1-p_j)]$$

$$\text{retrieval\_InfoNCE} = -\log\left(\frac{\exp(\text{sim}(q, d_{\text{pos}})/\text{temp})}{\sum_d \exp(\text{sim}(q, d)/\text{temp})}\right)$$

$$\text{pairwise\_rank\_loss} = -\log \text{sigmoid}(\text{score}(q, d_{\text{pos}}) - \text{score}(q, d_{\text{neg}}))$$

$$\text{best\_action} = \underset{\text{action}}{\text{argmin}} \sum_{\text{outcome}} P(\text{outcome} \mid X) * \text{cost}(\text{action}, \text{outcome})$$

## Multi-task option

A shared conversation encoder may support intent, risk, entity, resolution, and escalation heads, reducing inference cost and sharing representations. Risks include negative transfer, hard loss weighting, and rare safety tasks being overwhelmed. Separate models are simpler initially; multi-task learning is justified once sufficient labeled data and stable task definitions exist.

## What remains deterministic

Authentication, ownership, permissions, refund eligibility, valid state transitions, idempotency, financial reconciliation, mandatory approval, and regulatory restrictions are not model targets. Models may predict what is likely; deterministic services establish what is allowed and what actually occurred.

## 12. Online Serving Architecture and End-to-End Flow

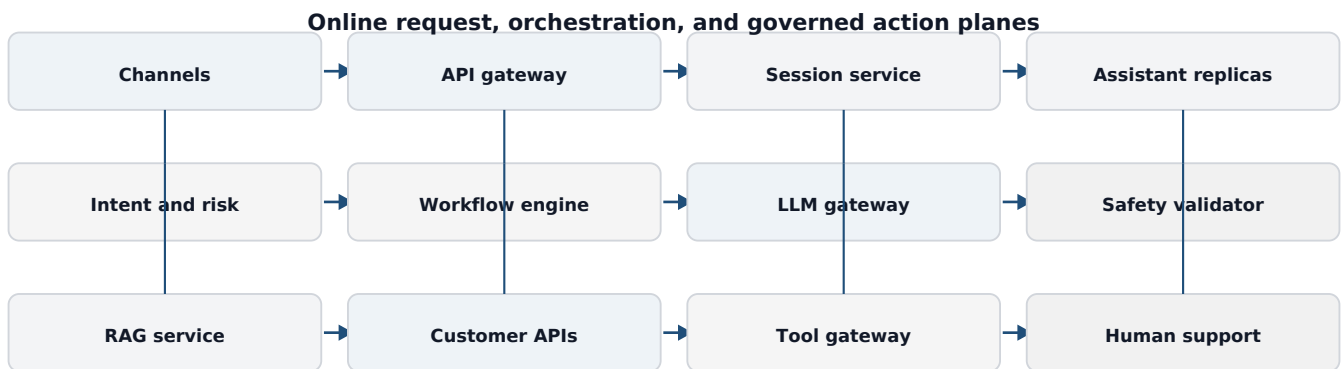


Figure 2. Vendor-neutral production architecture with independently scalable planes.

### Major components

| Component                | Responsibility                                                           |
|--------------------------|--------------------------------------------------------------------------|
| API gateway              | Authentication integration, quotas, routing, abuse controls, request IDs |
| Session and case service | Message ordering, deduplication, state snapshot, pending question        |
| Intent and risk router   | Choose knowledge, customer-state, workflow, or human path                |
| RAG service              | Rewrite, filter, hybrid retrieve, rerank, assemble evidence              |
| Workflow engine          | Allowed states, preconditions, confirmation, stopping rules              |
| Customer-state gateway   | Typed read-only access to operational domains                            |
| Tool gateway             | Schema validation, authorization context, deadlines, idempotency         |
| Policy engine            | Eligibility decisions with versioned reason codes                        |
| LLM gateway              | Model routing, quotas, timeout, cost, fallback, prompt version           |
| Safety validator         | Grounding, sensitive output, prompt injection, claim checks              |
| Human platform           | Queue routing, ownership, agent-assist, synchronized case state          |
| Event pipeline           | Audit, analytics, evaluation, summaries, offline training                |

### End-to-end turn

- 1 Authenticate the request, rate-limit by account, channel, tool, and token budget, then assign trace and message IDs.
- 2 Deduplicate and load the durable case snapshot, recent context, pending question, and human ownership state.
- 3 Classify intent, risk, required information source, and whether the request is mixed or ambiguous.
- 4 Run independent evidence retrieval and read-only customer-state calls in parallel when dependencies allow.
- 5 Update verified state, select a deterministic workflow, and ask a targeted clarification if a resource remains ambiguous.
- 6 For actions, validate schema, authorization, eligibility, and confirmation; execute with idempotency and concurrency control.

- 7 Generate from approved evidence and structured tool results, then validate material claims and sensitive output.
- 8 Persist events, state transition, response, pending operation, and commitments; stream only claims safe to reveal.
- 9 Predict resolution and decide whether to verify, wait, close provisionally, or hand off with a structured summary.

## Asynchronous work

Move analytics, low-priority summaries, emails, embedding updates, evaluation sampling, and CRM enrichment to durable queues. Workers use at-least-once delivery, idempotent processing, backoff, dead-letter queues, and per-resource ordering where needed. Transactional audit and business updates use an outbox or equivalent durable handoff before success is acknowledged.

## 13. Scalability, Latency, and Cost

### Capacity dimensions

```
request_rate = active_users * turns_per_user_per_second
```

```
token_throughput = request_rate * (input_tokens + output_tokens)
```

```
concurrent_generations ~= request_rate * average_generation_latency
```

One thousand active users sending one message per minute produce about 16.7 RPS under that specific assumption. If a generation averages six seconds, the model plane may need roughly 100 simultaneous generations. Capacity planning must also cover streaming connections, retrieval QPS, reranker pairs, tool calls, queue events, and correlated incident bursts.

### Independent scaling

| Subsystem              | Scaling signal             | Primary bottleneck                 |
|------------------------|----------------------------|------------------------------------|
| Gateway                | Connections and RPS        | Network and authentication         |
| Stateless orchestrator | Active turns               | CPU, memory, open streams          |
| Case store             | Reads/writes per case      | Hot partitions and consistency     |
| Search                 | Query QPS and corpus size  | Index memory and tail latency      |
| Reranker               | Query-passage pairs        | Accelerator throughput             |
| LLM gateway            | Tokens/sec and concurrency | Provider quota or serving capacity |
| Tool services          | Domain-specific QPS        | Downstream dependencies            |
| Workers                | Queue depth and oldest age | Backlog processing                 |
| Human support          | Escalation arrival rate    | Agent capacity and skills          |

### Illustrative latency budget

| Stage                | Typical target    | Optimization                                  |
|----------------------|-------------------|-----------------------------------------------|
| Gateway and state    | 20-100 ms         | Connection pooling, local cache, partitioning |
| Routing and rewrite  | 20-150 ms         | Small models and deterministic rules          |
| Retrieval and rerank | 100-400 ms        | Parallel search, ANN, batched cascade         |
| Customer read API    | 50-500 ms         | Regional proximity and bounded cache          |
| First model token    | 300-1,500 ms      | Reserved capacity, shorter context, routing   |
| Write tool           | 100 ms to seconds | Progress events, deadlines, reconciliation    |

### Critical-path optimization

- Run independent policy retrieval and customer-state reads in parallel; preserve order for dependent checks.
- Use smaller models for intent, rewriting, summaries, and simple FAQs; reserve larger models for ambiguity and complexity.
- Use structured state, a recent window, top reranked evidence, and only relevant tool schemas to control context size.
- Cache stable public retrieval by policy version; never share customer-specific answers across identities.

- Batch reranking and offline embeddings; cascade from a small reranker to a larger one only when uncertainty warrants it.
- Stream safe explanatory text and deterministic progress, but buffer claims about consequential actions until verified.

## Cost model

```
case_cost = LLM + retrieval + storage + tools + infrastructure + human_handling
```

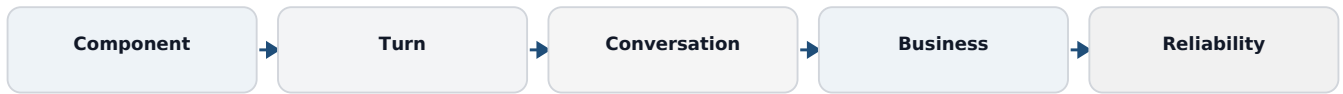
```
cost_per_successful_resolution = total_support_cost / correctly_resolved_cases
```

Reducing per-turn model cost can increase total case cost if poor answers cause recontact or long conversations. Early escalation is economically rational when ten low-probability automation turns would still end in human support.



# 14. Evaluation, Metrics, and Monitoring

## Evaluation hierarchy



## Model metrics

| Area       | Metrics                                               | Why                                              |
|------------|-------------------------------------------------------|--------------------------------------------------|
| Retrieval  | Recall@k, MRR, NDCG, coverage, freshness              | Correct evidence must be present and prioritized |
| Generation | Correctness, groundedness, claim coverage             | Fluent unsupported text is unsafe                |
| Tools      | Selection, argument, entity-binding, schema validity  | Wrong tool or resource creates harm              |
| Resolution | AUPRC, calibration, survival fit                      | Positive outcomes can be delayed and imbalanced  |
| Escalation | Precision, recall, cost-weighted error                | Both missed and unnecessary handoffs matter      |
| Memory     | Summary faithfulness, step repetition, state validity | Multi-turn correctness depends on state          |

## Business metrics

- Safe automated resolution and first-contact resolution.
- Same-issue recontact, customer satisfaction, customer effort, and abandonment.
- Human handling time after handoff, transfer acceptance, and repeated-question rate.
- Cost per successful resolution, support capacity released, and incorrect-compensation cost.

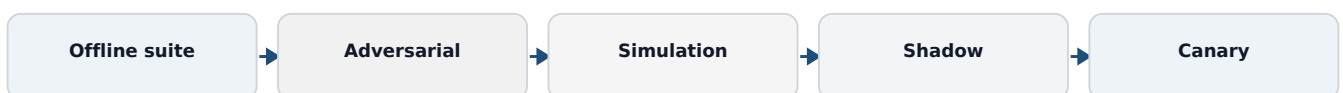
## System metrics

- P50/P95/P99 end-to-end latency, time to first token, active streams, QPS, and tokens/sec.
- Search and reranker latency, cache hit and stale-read rate, model throttling, and tool timeouts.
- Replica utilization, autoscaling delay, queue depth, oldest-event age, availability, and cost per case.

## Safety guardrails

Release-blocking metrics include unsupported claims, wrong policy, unauthorized disclosure, incorrect or duplicate action, prompt-injection success, cross-tenant leakage, missed mandatory escalation, and premature success claims. Authorization leakage and duplicate financial effects require effectively zero tolerance.

## Evaluation and rollout



Build versioned scenarios with initial state, customer turns, required facts, allowed tools, forbidden actions, expected transitions, and acceptable outcomes. Test common, long-tail, multilingual, ambiguous, no-answer, stale-policy, tool-failure, prompt-injection, and mandatory-escalation cases. Roll out from shadow to low-risk canary before higher-risk workflows.

## Problem-specific traces

A turn trace spans authentication, state read, intent, rewrite, filtering, retrieval, reranking, evidence, policy, tool, generation, validation, response, and state update. Redact secrets and personal data. Segment all quality and system metrics by intent, product, language, region, risk, model, prompt, retrieval index, workflow, and policy version.

# 15. Security, Reliability, and Failure Handling

## Prompt injection and data exfiltration

Injection can arrive through customer text, retrieved documents, CRM notes, tool output, uploads, and web content. Keep instructions separate from evidence, allowlist tools, enforce strict schemas, restrict network destinations, apply least privilege, validate resource ownership independently, minimize returned fields, redact secrets, and enforce workflows outside the model. An injection detector is an additional signal, not the primary security boundary.

## Reliability matrix

| Failure                     | Safe behavior                    | Fallback                                     |
|-----------------------------|----------------------------------|----------------------------------------------|
| Large model unavailable     | Do not silently reduce safety    | Validated small model for low-risk requests  |
| Retrieval unavailable       | Do not answer policy from memory | Deterministic FAQ or abstention              |
| Policy engine unavailable   | Block consequential action       | Human escalation                             |
| Customer API unavailable    | Do not claim live status         | Explain and create follow-up                 |
| Tool timeout                | Record UNKNOWN                   | Reconcile original operation                 |
| Reranker overloaded         | Preserve filters and authority   | Use fused first-stage ranking                |
| Cache unavailable           | Protect source of truth          | Read durable store with load shedding        |
| Queue backlog               | Preserve critical events         | Priority queues and worker scaling           |
| Streaming disconnect        | Avoid duplicate processing       | Resume from persisted message and case state |
| Repeated automation failure | Stop looping                     | Human handoff with trace                     |

## Operational patterns

- Deadlines and time budgets prevent one dependency from consuming the entire request SLA.
- Circuit breakers and bulkheads isolate failing model, search, and domain services.
- Exponential backoff with jitter retries only transient failures and retains the same operation identity.
- Load shedding preserves authenticated status and critical workflows before expensive open-ended generation.
- Transactional outbox and reconciliation close the gap between domain updates and emitted events.
- Versioned models, prompts, indexes, tools, workflows, and policies support canary rollout and rollback.

## Load and chaos testing

Test normal traffic, incident bursts, long conversations, large evidence contexts, simultaneous streams, cache misses, slow APIs, model throttling, tool retries, queue backlog, regional failure, and human-escalation surges. Load testing is iterative: identify a bottleneck, remediate, rerun, and observe where the bottleneck moves.

# 16. Alternatives, Trade-offs, and Production Evolution

## Major design choices

| Choice         | Selected approach                                  | Alternative and trade-off                                                                |
|----------------|----------------------------------------------------|------------------------------------------------------------------------------------------|
| Orchestration  | Deterministic workflows with bounded LLM proposals | Open-ended agents are flexible but harder to audit and constrain                         |
| Retrieval      | Hybrid BM25 plus dense plus reranking              | Dense-only is simpler but weak on identifiers and exact policy language                  |
| Memory         | Structured state plus bounded context              | Full history is simpler but grows cost and error surface quadratically                   |
| Tools          | Typed allowlisted APIs                             | Direct DB or arbitrary HTTP access is flexible but unsafe and tightly coupled            |
| Scaling        | Stateless orchestration with external state        | Sticky stateful servers simplify local memory but waste capacity and complicate recovery |
| Model strategy | Risk-aware model routing                           | One large model simplifies routing but increases cost and blast radius                   |
| Policies       | Deterministic versioned engine                     | LLM policy interpretation handles prose but is probabilistic and harder to audit         |
| Escalation     | Expected risk and resolution policy                | Fixed turn-count escalation ignores issue complexity and risk                            |

## Production evolution

- 1 Stage 0 - Human baseline: measure intent mix, resolution, article usage, handling time, recontact, and escalation.
- 2 Stage 1 - Search assistant: approved FAQ corpus, lexical retrieval, article links, no customer actions.
- 3 Stage 2 - Basic RAG: structure-aware chunks, dense search, citations, no-answer behavior, offline evaluation.
- 4 Stage 3 - Production RAG: metadata filters, hybrid retrieval, reranking, versioning, grounding, ACLs, autoscaling.
- 5 Stage 4 - Read-only personalization: authenticated customer APIs and structured case state.
- 6 Stage 5 - Controlled actions: deterministic workflows, policy engine, confirmation, idempotent tools, audits.
- 7 Stage 6 - Adaptive support: calibrated resolution and escalation, model routing, selective multi-task learning.
- 8 Stage 7 - Proactive support: incident-aware outreach and follow-up, governed by consent and risk controls.

## When complexity is justified

Add ANN only when exact search no longer meets scale or latency. Add reranking when candidate recall is acceptable but ordering limits answer quality. Add fine-tuning when stable repeated behavior cannot be achieved economically through prompting. Add autonomous planning only in bounded, low-risk, exploratory workflows. Every new component requires a measured failure it is intended to solve and an evaluation capable of detecting regression.

## 17. Final Architecture and Interview Review Checklist

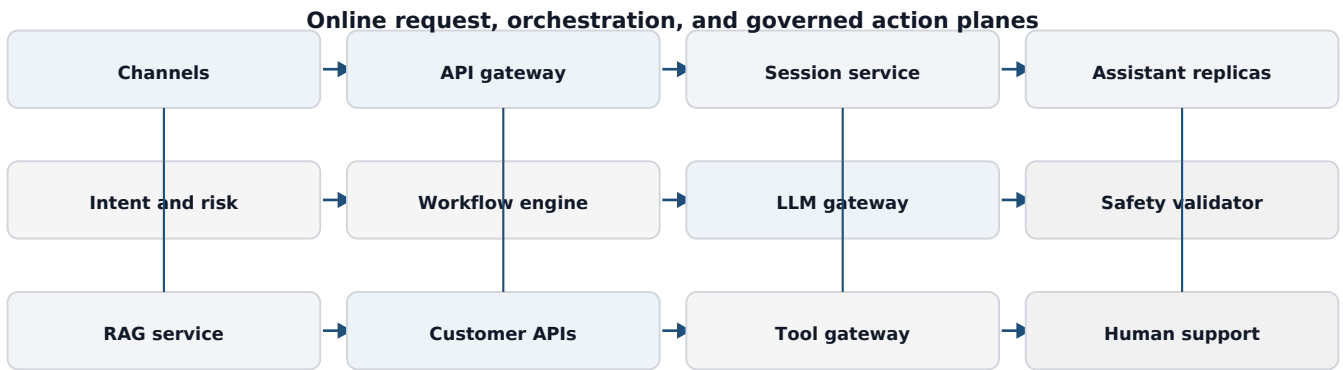


Figure 3. Final vendor-neutral architecture. Every layer exists to solve a specific quality, control, or scaling constraint.

### Final data and decision flow

The API gateway authenticates and protects downstream capacity. Stateless assistant replicas load external case state, route the request, and coordinate RAG, customer APIs, policy, tools, and models. The RAG service retrieves current authorized evidence. Domain services own operational state and valid transitions. The policy engine owns eligibility. The tool gateway executes idempotently. The safety layer validates claims and sensitive output. The event pipeline records the full decision path. Human support shares the same case state and receives ownership when automation should stop.

### Principal-level interview checklist

- Define the business objective as safe resolution with low customer effort, not maximum containment.
- Segment knowledge, live state, transactions, mixed requests, and high-risk cases before choosing models.
- Specify case states, pending confirmations, ownership, resolution signals, and point-in-time labels.
- Explain document authority, effective dates, ACLs, structure-aware chunks, hybrid retrieval, reranking, and sufficiency.
- Separate LLM proposal from authentication, authorization, eligibility, confirmation, execution, and reconciliation.
- Show idempotency, concurrent state changes, timeouts, UNKNOWN outcomes, outbox, retries, and human handoff.
- Use immutable events, verified structured state, recent context, summaries, and selective memory instead of full history.
- Choose answer, clarify, abstain, or escalate using component uncertainty, risk thresholds, and calibrated outcomes.
- Define specialized models, hard negatives, losses, temporal splits, delayed labels, selection bias, and feedback loops.
- Separate model, business, system, and safety metrics; connect offline evaluation to shadow, canary, and rollback.
- Calculate RPS, active generations, token throughput, retrieval QPS, downstream load, queue depth, and human capacity.
- Optimize cost per successful resolution, not token cost or response cost in isolation.

---

# Key takeaways

- 1. The assistant is a governed decision system, not a free-form chatbot.**
- 2. RAG supplies evidence; APIs supply current state; tools perform authorized actions.**
- 3. Deterministic controls own permission, policy, state transitions, idempotency, and money.**
- 4. Structured case state makes multi-turn support correct and scalable.**
- 5. Clarification, abstention, and escalation are core capabilities, not failures.**
- 6. The production objective is safe, correct resolution at acceptable latency and cost.**

Day 03 | Production-Grade AI Customer-Support Assistant  
Evidence-grounded. Tool-governed. Stateful. Calibrated. Scalable.